



UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Projeto de Software Web, Móvel e na Nuvem
Sistema de Gestão Digital para a Oficina
Duarte & Raposo

Tiago Dias Pereira - 55019

19 de julho de 2026

Conteúdo

Resumo	4
1 Introdução	5
1.1 Âmbito e Motivação	5
1.2 Objetivos da Implementação	5
2 Engenharia de Requisitos	6
2.1 Levantamento de Dados Físicos	6
2.2 Histórias de Utilizador (<i>User Stories</i>)	6
3 Modelação de Dados e Arquitetura	7
3.1 Modelo Relacional e Decisões de Desenho	7
3.2 A Escolha do Docker para Infraestrutura	8
4 Desenvolvimento da API REST (Backend)	9
4.1 Arquitetura RESTful (Node.js)	9
4.2 Rotas Implementadas	9
4.3 Decisões Técnicas de Implementação	10
4.4 Ambiente de Desenvolvimento Multiplataforma (Docker)	11
4.5 Trabalho a Realizar	11
5 Testes e Validação	12
5.1 Metodologia de Teste	12
5.2 Casos de Teste Executados	12
5.3 Validação de Integridade Referencial	12
6 Conclusão	13
7 Referências Bibliográficas	14
Acrónimos	14
8 Anexos	15
Anexos	15

Resumo

Este relatório descreve o desenvolvimento integral de um sistema de gestão desenhado à medida para a oficina automóvel Duarte & Raposo, no âmbito da unidade curricular de Projeto de Software Web, Móvel e na Nuvem. O objetivo prático é resolver um problema real com o qual lido diariamente no meu local de trabalho: a ineficiência operacional, a desorganização e a perda de histórico geradas pelo uso exclusivo de folhas de obra em papel.

O trabalho abrangeu todo o ciclo de vida do sistema: desde a engenharia de requisitos e modelação concetual, passando pela normalização do esquema lógico relacional (3FN) e implementação num SGBD SQL Server via Docker, até à construção de uma Aplicação Cliente (PWA e Dashboard) suportada por uma API em Node.js. A solução permite agora a gestão digital de veículos complexos (como autocaravanas), auditoria de colaboradores e processamento ágil para faturação.

Palavras-chave

Informática Web; React; Node.js; SQL Server; PWA; Docker; Engenharia de Requisitos; Autocaravanas.

1 Introdução

1.1 Âmbito e Motivação

No dia a dia da oficina Duarte & Raposo, especializada em eletricidade, mecânica e autocaravanas, todo o processo de entrada de veículos e registo de peças é feito de forma manual. Este sistema baseado em papel tem várias falhas críticas: é difícil consultar o histórico de uma autocaravana que já lá esteve no ano passado, os papéis sujam-se ou perdem-se pela oficina, e no final da reparação perde-se muito tempo a fazer contas à mão para conseguir passar a fatura ao cliente.

A motivação para este projeto surge exatamente da vontade de juntar o útil ao agradável: aplicar os conhecimentos adquiridos ao longo do curso de Informática Web para desenvolver uma solução real que facilite não só o meu trabalho, mas o de toda a equipa. O foco vai estar especialmente nas autocaravanas, dado que são veículos complexos (com parte mecânica e parte habitacional) e exigem um registo mais detalhado que um veículo ligeiro comum.

1.2 Objetivos da Implementação

- **Acabar com o papel:** Passar todo o registo de clientes, veículos e folhas de obra para uma base de dados digital e relacional, garantindo a integridade dos dados.
- **Aplicação Móvel (PWA):** Desenvolver uma interface em **React** leve e direta, para que qualquer mecânico consiga dar entrada a um carro no sistema usando o telemóvel, sem perder muito tempo.
- **Gestão no Computador:** Criar uma interface web *desktop* (Dashboard) para o Duarte e o Raposo poderem ver o que está a ser feito, consultar históricos e ver as contas automáticas do IVA para a faturação.
- **Arquitetura e Portabilidade:** Construir o *backend* em Node.js com base de dados Structured Query Language (SQL) Server, e utilizar o **Docker** para contentorizar o projeto. Isto vai garantir que o sistema corre sempre bem, quer esteja a programar no meu computador fixo ou num portátil novo, sem dores de cabeça com compatibilidades.

2 Engenharia de Requisitos

2.1 Levantamento de Dados Físicos

Como trabalho diretamente na oficina, o levantamento de requisitos foi muito direto e baseado na realidade operacional. Peguei nas folhas de obra e faturas em branco que usamos atualmente e retirei a informação essencial que tem de transitar para a plataforma digital:

- **Dados do Cliente:** Nome, Morada, NIF e Telefone.
- **Dados do Veículo:** Matrícula, e, no caso das autocaravanas, a separação imperativa entre a Marca/Modelo do Chassis e a Marca da Célula.
- **Dados da Entrada:** Data, os quilómetros (KMS) com que o carro entrou, e espaço para observações gerais.
- **O Serviço:** As várias linhas de material gasto e horas de trabalho, com quantidade, preço unitário e categoria do serviço.

2.2 Histórias de Utilizador (*User Stories*)

Com base nas rotinas da equipa e nas dificuldades sentidas no "chão de oficina", estruturei as necessidades interativas do sistema:

- *"Como mecânico, preciso de abrir a app no telemóvel e registar a entrada de um carro em poucos cliques, para voltar rapidamente ao trabalho."*
- *"Como mecânico, quero adicionar as peças e serviços que usei diretamente à folha de obra ativa daquele carro no exato momento em que as aplico."*
- *"Como mecânico, preciso de uma secção de 'Conselhos' para avisar o cliente sobre peças que vão precisar de ser trocadas no futuro, para prestarmos um bom serviço preventivo."*
- *"Como gestor, preciso que o sistema some os valores do material, faça o cálculo do IVA sozinho quando a reparação acaba, e identifique quem foi o mecânico responsável pela obra."*

3 Modelação de Dados e Arquitetura

3.1 Modelo Relacional e Decisões de Desenho

Para garantir a integridade referencial dos dados e mitigar as redundâncias presentes no modelo de papel (como escrever o nome do cliente dez vezes para o mesmo carro ao longo do ano), desenhei uma base de dados relacional (ver Anexos) que cumpre com os normativos da Terceira Forma Normal (3NF).

Durante esta fase, tomei as seguintes opções técnicas para adaptar o sistema à realidade do negócio:

- **Separação Estrutural das Autocaravanas:** Decidi modelar a entidade *Autocaravana* separando explicitamente a parte mecânica (**Marca_chassis**, **Modelo_chassis**) da parte habitacional (**marca_celula**). Isto é vital na nossa oficina, pois permite encomendar peças de motor à Fiat/Ford e peças de habitáculo a fornecedores de campismo de forma isolada e precisa.
- **Rastreabilidade Operacional (*Auditing*):** No papel, as assinaturas perdem-se. Para resolver isto, criei a entidade *Colaboradores*. Foi estabelecido um relacionamento onde cada *Folha_Obra* guarda obrigatoriamente a Chave Estrangeira do colaborador (**ID_Colaborador**) que a registou, garantindo auditoria e responsabilização interna.
- **Preservação Fisco-Operacional (Soft Delete):** Numa empresa real, apagar dados com um DELETE pode destruir o histórico fiscal e garantias de peças. Para acautelar isto, implementei um sistema de *Soft Delete*: adicionei o atributo **ativo** (booleano) às tabelas principais (Clientes, Veículos, Colaboradores). Os dados nunca são eliminados da base de dados, são apenas ocultados da interface.

O esquema relacional consolida-se assim em cinco entidades principais:

1. **Colaboradores:** ID_Colaborador, Nome, Cargo, Email, Password_Hash, Ativo.
2. **Clientes:** ID_Cliente, Nome, Contribuinte_NIF, Telefone, Morada, Ativo.
3. **Autocaravanas:** Matricula, Marca_Chassis, Modelo_Chassis, Marca_Celula, Ano, Ativo, *ID_Cliente (FK)*.
4. **Folhas_Obra:** ID_Folha, Data_Entrada, KMS_Entrada, Estado_Reparacao, Observacoes, Conselhos, *Matricula (FK)*, *ID_Colaborador (FK)*.
5. **Linhas_Reparacao:** ID_Linha, Designacao, Quantidade, Valor_Unitario, Categoria, *ID_Folha (FK)*.

3.2 A Escolha do Docker para Infraestrutura

Em termos de arquitetura de servidor, decidi utilizar o **Docker** neste projeto, alinhando a solução com as melhores práticas de *DevOps* lecionadas no curso.

A vantagem principal para o meu caso de uso é a facilidade de criar ambientes isolados. Ao definir um ficheiro `docker-compose.yml`, consigo ter a base de dados SQL Server e, futuramente, a API em Node.js a correr em qualquer computador através de um único comando. Isto evita que o meu computador fique "sujo" com serviços em segundo plano, consome menos recursos, e facilita drasticamente o *deploy* do projeto para um servidor de produção quando a oficina adotar o sistema em definitivo.

4 Desenvolvimento da API REST (Backend)

Esta secção documenta o estado atual da implementação do *backend*, desenvolvido em Node.js. À data de escrita desta secção, encontram-se implementadas as três entidades independentes do modelo (Colaboradores, Clientes e Autocaravanas); as entidades relacionais mais complexas (Folhas de Obra e Linhas de Reparação) e a camada de interface (PWA e Dashboard) estão planeadas para a fase seguinte do projeto (ver secção 4.5).

4.1 Arquitetura RESTful (Node.js)

No desenvolvimento da camada de *backend*, optou-se pela utilização do ambiente Node.js com a *framework* Express. Esta API atua como o motor central de regras de negócio, comunicando diretamente com o SQL Server e expondo os *endpoints* necessários para as operações Create, Read, Update, Delete (CRUD).

O código foi organizado de forma modular, separando claramente três responsabilidades:

- `db.js` — gestão centralizada da ligação à base de dados;
- `routes/` — um ficheiro por entidade (`colaboradores.js`, `clientes.js`, `autocaravanas.js`), cada um definindo as suas próprias rotas através do `express.Router()`;
- `server.js` — ponto de entrada da aplicação, responsável apenas por montar as rotas e arrancar o servidor.

Esta separação, em vez de concentrar toda a lógica num único ficheiro, torna o código mais fácil de expandir à medida que novas entidades (como as Folhas de Obra) são adicionadas, e facilita a leitura do projeto em contexto de avaliação académica.

4.2 Rotas Implementadas

A Tabela 1 resume os *endpoints* atualmente disponíveis. Todas as entidades seguem o mesmo padrão CRUD, com a particularidade de o método DELETE nunca remover fisicamente um registo (ver *Soft Delete*, secção 3.1).

Método	Endpoint	Descrição
GET	/api/colaboradores	Lista colaboradores ativos
GET	/api/colaboradores/:id	Obtém um colaborador
POST	/api/colaboradores	Cria colaborador (password encriptada com bcrypt)
PUT	/api/colaboradores/:id	Atualiza colaborador
DELETE	/api/colaboradores/:id	Desativa colaborador (soft delete)
GET	/api/clientes	Lista clientes ativos
GET	/api/clientes/:id	Obtém um cliente
POST	/api/clientes	Cria cliente
PUT	/api/clientes/:id	Atualiza cliente
DELETE	/api/clientes/:id	Desativa cliente (soft delete)
GET	/api/autocaravanas	Lista autocaravanas ativas (com nome do cliente)
GET	/api/autocaravanas/:matricula	Obtém uma autocaravana
POST	/api/autocaravanas	Cria autocaravana (valida existência do cliente)
PUT	/api/autocaravanas/:matricula	Atualiza autocaravana
DELETE	/api/autocaravanas/:matricula	Desativa autocaravana (soft delete)

Tabela 1: Rotas da API implementadas até ao momento.

4.3 Decisões Técnicas de Implementação

- **Pool de Ligações Único:** Em vez de abrir uma nova ligação ao SQL Server em cada pedido HTTP (`sql.connect()` por rota), foi implementado um pool de ligações único e partilhado por toda a aplicação (`db.js`), seguindo a recomendação da própria biblioteca `mssql`. Isto evita o esgotamento de ligações disponíveis sob carga.
- **Encriptação de *Passwords*:** As *passwords* dos colaboradores nunca são guardadas em texto simples. Antes de qualquer escrita na base de dados, são processadas com a biblioteca `bcrypt` (`salt rounds = 10`), preparando o terreno para a futura implementação de autenticação (*login*) na Fase 2.
- **Validação de Integridade Referencial ao Nível da Aplicação:** Antes de criar uma Autocaravana, a API confirma explicitamente que o `ID_cliente` indicado existe e está ativo, devolvendo um erro 400 claro em caso negativo. Sem esta validação, um erro de chave estrangeira do SQL Server chegaria ao cliente da API de forma menos perceptível.

- **Tratamento de Duplicados:** Os códigos de erro nativos do SQL Server para violação de restrições **UNIQUE** (2627/2601) são capturados e traduzidos para respostas **HTTP 409 Conflict** com mensagens em português, em vez de expor o erro técnico da base de dados diretamente.

4.4 Ambiente de Desenvolvimento Multiplataforma (Docker)

Um aspeto prático relevante do desenvolvimento foi garantir que o ambiente Docker funciona de forma consistente em diferentes máquinas de desenvolvimento, nomeadamente entre um computador com processador Intel e um MacBook com Apple Silicon (M1/M2/M3/M4). A imagem oficial do SQL Server é distribuída apenas para arquitetura x86_64, pelo que em Macs Apple Silicon a execução ocorre por emulação (Rosetta 2). Para tornar este comportamento explícito e evitar erros de arquitetura, foi adicionada a diretiva `platform: linux/amd64` ao serviço da base de dados no `docker-compose.yml`, garantindo o arranque correto do container independentemente do processador do anfitrião.

4.5 Trabalho a Realizar

Ficam planeados para a fase seguinte do projeto:

- Implementação das rotas de **Folhas de Obra** e **Linhas de Reparação**, que exigem lógica transacional (`sql.Transaction`) para garantir que a folha de obra e as suas linhas associadas são criadas de forma atómica;
- Implementação de **autenticação** (*login* de colaboradores via JSON Web Token (JWT), aproveitando o campo `PasswordHash` já existente);
- Desenvolvimento da **PWA em React** para os mecânicos (visão mobile);
- Desenvolvimento do **Dashboard em React** para a gestão (visão desktop), incluindo o cálculo automático de IVA para faturação.

5 Testes e Validação

5.1 Metodologia de Teste

Os testes realizados até ao momento são testes manuais de integração, executados através da ferramenta *Thunder Client*¹ e da linha de comandos (*curl*), validando o comportamento de cada rota diretamente contra a base de dados SQL Server a correr em Docker. A automatização de testes (por exemplo, com *Jest* e *Supertest*) fica identificada como possível extensão para a fase seguinte.

5.2 Casos de Teste Executados

A Tabela 2 resume os cenários validados manualmente para as três entidades implementadas.

Cenário	Resultado Esperado	OK?
Criar cliente com todos os campos válidos	Resposta 201, registo devolvido com <code>ID_cliente</code>	Sim
Listar clientes	Resposta 200, apenas clientes com <code>Ativo = 1</code>	Sim
Criar autocaravana com <code>id_cliente</code> inexistente	Resposta 400, mensagem clara (sem expor erro SQL)	Sim
Criar autocaravana com matrícula já existente	Resposta 409 Conflict	Sim
Criar colaborador e confirmar que a <i>password</i> não é devolvida em texto simples	Resposta 201, campo <code>password</code> ausente da resposta	Sim
Desativar (soft delete) um colaborador e tentar listá-lo	Não aparece na listagem, mas mantém-se na base de dados	Sim

Tabela 2: Casos de teste manuais executados sobre a API.

5.3 Validação de Integridade Referencial

Foram realizadas tentativas propositadas de inserção de dados inconsistentes para confirmar que o sistema bloqueia operações ilícitas — por exemplo, associar uma autocaravana a um `ID_cliente` inexistente ou já inativo.

¹Alternativamente, poderia ser usado o Postman; a escolha não afeta os resultados.

Estas validações são feitas explicitamente ao nível da aplicação (e não apenas por rejeição da base de dados), permitindo devolver mensagens de erro perceptíveis ao utilizador final da API.

[Nota de Edição: inserir aqui as capturas de ecrã do Thunder Client/Postman correspondentes aos testes da Tabela 2, uma por cada pedido relevante, por exemplo:]

6 Conclusão

[Nota de Edição: esta é uma síntese do estado intercalar do projeto; será revista e completada no relatório final, após a conclusão das Folhas de Obra, autenticação e interfaces React.]

O desenvolvimento deste projeto, até ao momento, percorreu com sucesso as fases de levantamento de requisitos, modelação de dados e implementação inicial do *backend*, tendo o modelo relacional e a infraestrutura Docker sido validados na prática.

O que foi conseguido até ao momento: Foi desenhado e fundamentado um modelo de dados relacional adaptado à realidade oficial, com foco nas autocaravanas, e implementado o *backend* correspondente às três entidades independentes do modelo (Colaboradores, Clientes e Autocaravanas), com CRUD completo, validações de integridade referencial e testes manuais documentados na secção anterior. A arquitetura *containerizada* em Docker provou ser eficaz e portátil entre diferentes máquinas de desenvolvimento.

O que falta concluir: as rotas de Folhas de Obra e Linhas de Reparação (com lógica transacional), a autenticação de colaboradores, e a camada de interface — a PWA em React para os mecânicos e o Dashboard para a gestão, incluindo o cálculo automático de IVA para faturação.

De uma forma geral, os objetivos propostos na fase de idealização estão a ser cumpridos progressivamente, com o *backend* a substituir já parte do fluxo em papel. A conclusão integral do sistema, com a camada de interface funcional, permitirá validar por completo a solução junto da oficina Duarte & Raposo.

7 Referências Bibliográficas

- Documentação Oficial Docker. *Docker Compose Overview*. Disponível em: <https://docs.docker.com/compose/>
- Documentação Microsoft. *SQL Server on Linux*. Disponível em: <https://learn.microsoft.com/en-us/sql/linux/>
- MDN Web Docs. *Progressive Web Apps (PWAs)*. Disponível em: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

Acrónimos

UBI Universidade da Beira Interior

API Application Programming Interface

SQL Structured Query Language

PWA Progressive Web App

CRUD Create, Read, Update, Delete

SGBD Sistema de Gestão de Bases de Dados

3FN Terceira Forma Normal

JWT JSON Web Token

8 Anexos

Anexo I: Diagrama da Base de Dados

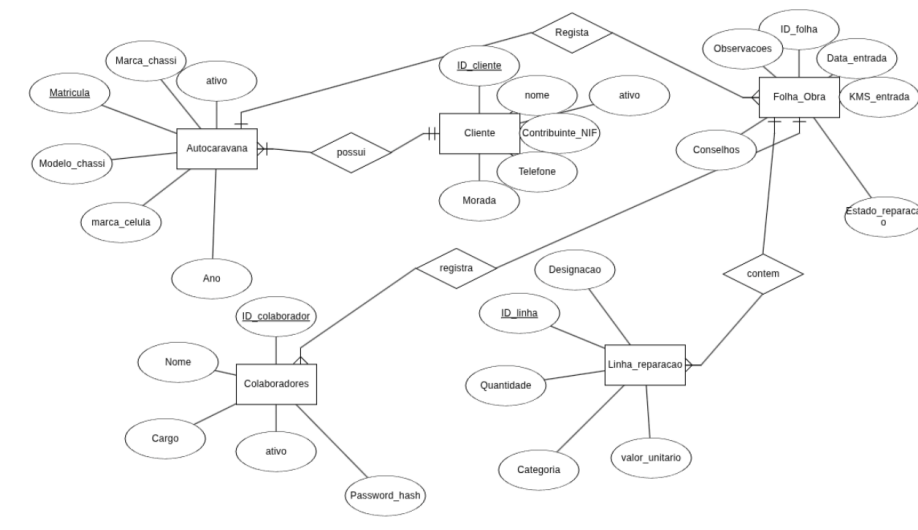


Figura 1: Diagrama concetual gerado no ERDPlus.

Anexo II: Esquema Relacional da Base de Dados

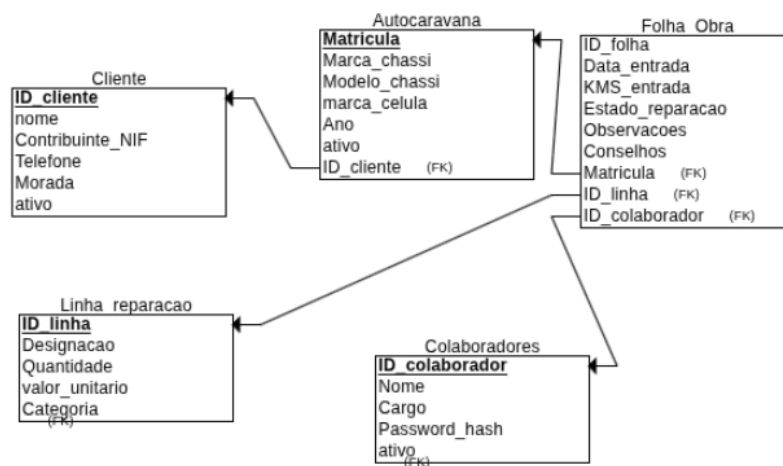


Figura 2: Esquema Relacional final gerado no ERDPlus.